

Zaawansowane metody tablic w JavaScript

Spis treści

1. Iteracja – <code>forEach()</code>	3
Składnia	3
Przykłady użycia <code>forEach()</code>	3
1. Wyświetlanie elementów tablicy	3
2. Iteracja z użyciem indeksu	3
3. Modyfikacja elementów (bez zmiany tablicy)	4
4. Zmiana wartości w tablicy (niezalecane)	4
Kiedy warto używać <code>forEach()</code> ?	4
Kiedy NIE używać <code>forEach()</code> ?	4
2. Mapowanie – <code>map()</code>	4
Składnia	4
Przykłady użycia	5
1. Podnoszenie liczb do kwadratu	5
2. Konwersja stopni Celsjusza na Fahrenheita	5
3. Dodawanie tekstu do każdego elementu	5
4. Ekstrakcja jednej właściwości z obiektów	5
Kiedy warto używać <code>map()</code> ?	5
3. Filtracja – <code>filter()</code>	5
Składnia	6
Przykłady użycia	6
1. Filtracja liczb większych niż 10	6
2. Znalezienie parzystych liczb	6
3. Filtracja obiektów – wybór pełnoletnich osób	6
4. Filtrowanie słów dłuższych niż 5 liter	6
Kiedy warto używać <code>filter()</code> ?	6
4. Redukcja – <code>reduce()</code>	7
Składnia	7
Przykłady użycia <code>reduce()</code>	7
1. Sumowanie liczb w tablicy	7
2. Obliczanie iloczynu liczb	7
3. Znalezienie największej liczby	8

4. Grupowanie danych w obiekt	8
5. Konkatenacja (łączenie) ciągów znaków.....	8
Kiedy warto używać reduce()?	8
5. Sortowanie – sort()	8
Składnia	9
Przykłady użycia sort()	9
1. Sortowanie tekstu (domyślnie jako stringi)	9
2. Sortowanie liczb (błąd w domyślnym sortowaniu)	9
3. Poprawne sortowanie liczb (z funkcją porównującą)	9
4. Sortowanie liczb malejąco	10
5. Sortowanie obiektów (np. według wieku)	10
Kiedy warto używać sort()?	10
Kiedy NIE używać sort()?	10
Podsumowanie	10
6. Odwracanie kolejności – reverse()	10
Składnia	11
Przykłady użycia reverse()	11
1. Odwrócenie kolejności elementów w tablicy.....	11
2. Odwrócenie kolejności znaków w stringu	11
3. Odwrócenie posortowanej tablicy (sortowanie malejące)	11
Kiedy warto używać reverse()?	11
Kiedy NIE używać reverse()?	12
Podsumowanie	12

1. Iteracja – forEach()

Metoda **forEach()** pozwala na wykonanie określonej funkcji dla każdego elementu tablicy. Nie zwraca nowej tablicy i **nie modyfikuje oryginalnej tablicy**.

Składnia

```
tablica.forEach((element, indeks, tablica) => {  
    // operacje na elementach  
});
```

- **element** – aktualny element tablicy.
- **indeks** (opcjonalnie) – indeks bieżącego elementu.
- **tablica** (opcjonalnie) – cała tablica, na której działa forEach().

Przykłady użycia forEach()

1. Wyświetlanie elementów tablicy

```
const fruits = ["jabłko", "banan", "gruszka", "śliwka"];  
  
fruits.forEach(fruit => {  
    console.log(fruit);  
});  
// Wynik:  
// jabłko  
// banan  
// gruszka  
// śliwka
```

forEach() przechodzi przez każdy element i wyświetla go w konsoli.

2. Iteracja z użyciem indeksu

```
const fruits = ["jabłko", "banan", "gruszka"];  
  
fruits.forEach((fruit, index) => {  
    console.log(`Indeks ${index}: ${fruit}`);  
});  
// Wynik:  
// Indeks 0: jabłko  
// Indeks 1: banan  
// Indeks 2: gruszka
```

forEach() umożliwia dostęp do indeksów.

3. Modyfikacja elementów (bez zmiany tablicy)

```
const liczby = [1, 2, 3, 4];

liczby.forEach((num, index) => {
  console.log(`Liczba: ${num}, podwojona: ${num * 2}`);
});

// Wynik:
// Liczba: 1, podwojona: 2
// Liczba: 2, podwojona: 4
// Liczba: 3, podwojona: 6
// Liczba: 4, podwojona: 8
```

Oryginalna tablica **nie zmienia się**, `forEach()` tylko wykonuje operację.

4. Zmiana wartości w tablicy (niezalecane)

Można modyfikować elementy tablicy wewnątrz `forEach()`, ale **nie jest to zalecana praktyka** (lepiej użyć `map()`).

```
const numbers = [1, 2, 3, 4];

numbers.forEach((num, index, arr) => {
  arr[index] = num * 2;
});

console.log(numbers); // [2, 4, 6, 8]
```

`forEach()` **może zmieniać tablicę**, ale nie zwraca nowej.

Kiedy warto używać `forEach()`?

- ✓ Gdy **chcesz wykonać operację na każdym elemencie tablicy**, ale **nie potrzebujesz nowej tablicy**.
- ✓ Gdy **chcesz wyświetlić elementy** lub **wykonać kod dla każdego z nich**.
- ✓ Gdy chcesz **przetwarzać dane**, ale **nie zmieniać struktury tablicy**.

Kiedy **NIE** używać `forEach()`?

- ✗ Jeśli chcesz utworzyć nową tablicę – użyj `map()`.
- ✗ Jeśli chcesz filtrować dane – użyj `filter()`.
- ✗ Jeśli chcesz uzyskać jedną wartość (np. sumę) – użyj `reduce()`.

2. Mapowanie – `map()`

Tworzy nową tablicę, przekształcając każdy element według podanej funkcji.

Składnia

```
const nowaTablica = staraTablica.map(element => wyrażenie);
```

- `map()` przechodzi przez każdy element `staraTablica`,

- wykonuje na nim podaną funkcję (np. przemnażanie, zmiana tekstu, itp.),
- zwraca nową tablicę z przekształconymi wartościami.

Przykłady użycia

1. Podnoszenie liczb do kwadratu

```
const liczby = [1, 2, 3, 4, 5];
const kwadraty = liczby.map(num => num * num);

console.log(kwadraty); // [1, 4, 9, 16, 25]
```

2. Konwersja stopni Celsjusza na Fahrenheita

```
const celsius = [0, 10, 20, 30];
const fahrenheit = celsius.map(temp => (temp * 9/5) + 32);

console.log(fahrenheit); // [32, 50, 68, 86]
```

3. Dodawanie tekstu do każdego elementu

```
const imiona = ["Anna", "Kuba", "Ola"];
const powitania = imiona.map(imie => `Cześć, ${imie}!`);

console.log(powitania);
// ["Cześć, Anna!", "Cześć, Kuba!", "Cześć, Ola!"]
```

4. Ekstrakcja jednej właściwości z obiektów

```
const osoby = [
  { imie: "Tomek", wiek: 25 },
  { imie: "Magda", wiek: 30 },
  { imie: "Kasia", wiek: 22 }
];

const imiona = osoby.map(osoba => osoba.imie);

console.log(imiona); // ["Tomek", "Magda", "Kasia"]
```

Kiedy warto używać map()?

- Gdy chcesz **utworzyć nową tablicę**, przekształcając elementy oryginalnej tablicy.
- Gdy **nie chcesz modyfikować** oryginalnej tablicy (jak robi np. forEach()).
- Gdy **chcesz zmienić format danych**, np. liczb na tekst, obiektów na tablicę wartości.

3. Filtracja – filter()

Metoda **filter()** tworzy **nową tablicę** zawierającą tylko te elementy, które spełniają określony warunek. **Nie modyfikuje** oryginalnej tablicy.

Składnia

```
const nowaTablica = staraTablica.filter(element => warunek);
```

- filter() przechodzi przez każdy element staraTablica,
- sprawdza, czy element spełnia podany warunek,
- jeśli tak – dodaje go do nowej tablicy,
- zwraca nową tablicę zawierającą tylko pasujące elementy.

Przykłady użycia

1. Filtracja liczb większych niż 10

```
const liczby = [5, 12, 8, 130, 44];

const wiekszeNiz10 = liczby.filter(num => num > 10);

console.log(wiekszeNiz10); // [12, 130, 44]
```

2. Znalezienie parzystych liczb

```
const liczby = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

const parzyste = liczby.filter(num => num % 2 === 0);

console.log(parzyste); // [2, 4, 6, 8, 10]
```

3. Filtracja obiektów – wybór pełnoletnich osób

```
const osoby = [
  { imie: "Tomek", wiek: 25 },
  { imie: "Magda", wiek: 17 },
  { imie: "Kasia", wiek: 19 }
];

const pelnoletni = osoby.filter(osoba => osoba.wiek >= 18);

console.log(pelnoletni);
// [{ imie: "Tomek", wiek: 25 }, { imie: "Kasia", wiek: 19 }]
```

4. Filtrowanie słów dłuższych niż 5 liter

```
const slowa = ["kot", "samochód", "pies", "domek", "programowanie"];

const dlugieSlowa = slowa.filter(word => word.length > 5);

console.log(dlugieSlowa); // ["samochód", "programowanie"]
```

Kiedy warto używać filter()?

- ✓ Gdy chcesz **utworzyć nową tablicę** zawierającą tylko elementy spełniające dany warunek.

- ✓ Gdy potrzebujesz **odfiltrować dane** (np. tylko liczby parzyste, tylko obiekty z daną wartością).
- ✓ Gdy chcesz **usunąć niechciane elementy**, ale **bez zmiany oryginalnej tablicy**.

4. Redukcja – reduce()

Metoda **reduce()** pozwala **zredukować tablicę** do jednej wartości (np. sumy, iloczynu, obiektu, łańcucha znaków itp.), przetwarzając jej elementy w sposób iteracyjny.

Składnia

```
const wynik = tablica.reduce((akumulator, element, indeks, tablica) => {  
  // operacja na elementach  
  return nowaWartośćAkumulatora;  
}, wartośćPoczątkowa);
```

- **akumulator** – przechowuje skumulowany wynik operacji.
- **element** – bieżący element tablicy.
- **indeks** (opcjonalne) – indeks aktualnie przetwarzanego elementu.
- **tablica** (opcjonalne) – oryginalna tablica, na której działa reduce().
- **wartośćPoczątkowa** (opcjonalna) – początkowa wartość akumulatora.

Przykłady użycia reduce()

1. Sumowanie liczb w tablicy

```
const liczby = [1, 2, 3, 4, 5];  
  
const suma = liczby.reduce((acc, num) => acc + num, 0);  
  
console.log(suma); // 15
```

reduce() sumuje liczby, zaczynając od 0 jako wartości początkowej.

2. Obliczanie iloczynu liczb

```
const liczby = [2, 3, 4];  
  
const iloczyn = liczby.reduce((acc, num) => acc * num, 1);  
  
console.log(iloczyn); // 24 (2 * 3 * 4)
```

reduce() mnoży kolejne liczby, zaczynając od wartości 1.

3. Znalezienie największej liczby

```
const liczby = [10, 5, 20, 8];

const max = liczby.reduce((acc, num) => (num > acc ? num : acc), liczby[0]);

console.log(max); // 20
```

`reduce()` przechodzi przez tablicę i porównuje kolejne liczby, zapisując największą.

4. Grupowanie danych w obiekt

Założmy, że mamy tablicę obiektów i chcemy policzyć, ile razy występują różne kategorie.

```
const produkty = [
  { nazwa: "Jabłko", kategoria: "Owoce" },
  { nazwa: "Banan", kategoria: "Owoce" },
  { nazwa: "Marchewka", kategoria: "Warzywa" },
  { nazwa: "Brokuł", kategoria: "Warzywa" }
];

const grupy = produkty.reduce((acc, produkt) => {
  acc[produkt.kategoria] = (acc[produkt.kategoria] || 0) + 1;
  return acc;
}, {});

console.log(grupy);
// { Owoce: 2, Warzywa: 2 }
```

`reduce()` tworzy obiekt liczący, ile razy dana kategoria się pojawiła.

5. Konkatenacja (łączenie) ciągów znaków

```
const slowa = ["Cześć", "jestem", "JavaScript"];

const zdanie = slowa.reduce((acc, slowo) => acc + " " + slowo, "");

console.log(zdanie); // "Cześć jestem JavaScript"
```

`reduce()` łączy słowa w jedno zdanie.

Kiedy warto używać `reduce()`?

- ✓ Gdy chcesz **przetworzyć tablicę do jednej wartości** (suma, iloczyn, obiekt, string itp.).
- ✓ Gdy **chcesz unikać pętli** (`for` lub `forEach()`).
- ✓ Gdy musisz **grupować, zliczać elementy lub wyciągać wartości** z tablicy.

5. Sortowanie – `sort()`

Metoda **`sort()`** służy do sortowania elementów tablicy **bez tworzenia nowej tablicy** – modyfikuje istniejącą tablicę.

Domyślnie sortuje elementy jako **ciągi znaków (stringi)**, co może prowadzić do nieoczekiwanych wyników przy liczbach.

Składnia

```
tablica.sort((a, b) => warunek);
```

- **a i b** – dwa porównywane elementy.
- Funkcja zwrotna (callback) powinna zwrócić:
 - **Liczbę ujemną (< 0)** – jeśli a powinno być przed b.
 - **Liczbę dodatnią (> 0)** – jeśli b powinno być przed a.
 - **Zero (0)** – jeśli kolejność nie ma znaczenia.

Przykłady użycia sort()

1. Sortowanie tekstu (domyślnie jako stringi)

```
const owoce = ["banan", "jabłko", "śliwka", "gruszka"];

owoce.sort();

console.log(owoce);
// ["banan", "gruszka", "jabłko", "śliwka"]
```

sort() sortuje alfabetycznie.

2. Sortowanie liczb (błąd w domyślnym sortowaniu)

```
const liczby = [10, 5, 20, 1, 100];

liczby.sort();

console.log(liczby);
// [1, 10, 100, 20, 5] ❌ (bo traktuje liczby jako stringi)
```

sort() **domyślnie traktuje liczby jak stringi** i sortuje je według pierwszej cyfry!

3. Poprawne sortowanie liczb (z funkcją porównującą)

```
const liczby = [10, 5, 20, 1, 100];

liczby.sort((a, b) => a - b); // Sortowanie rosnące

console.log(liczby);
// [1, 5, 10, 20, 100]
```

Funkcja a - b sortuje liczby **w kolejności rosnącej**.

4. Sortowanie liczb malejąco

```
const liczby = [10, 5, 20, 1, 100];

liczby.sort((a, b) => b - a); // Sortowanie malejące

console.log(liczby);
// [100, 20, 10, 5, 1]
```

b - a powoduje sortowanie **od największej do najmniejszej**.

5. Sortowanie obiektów (np. według wieku)

```
const osoby = [
  { imie: "Tomek", wiek: 25 },
  { imie: "Magda", wiek: 30 },
  { imie: "Kasia", wiek: 22 }
];

osoby.sort((a, b) => a.wiek - b.wiek);

console.log(osoby);
// [{ imie: "Kasia", wiek: 22 }, { imie: "Tomek", wiek: 25 }, { imie: "Magda", wiek: 30 }]
```

Sortowanie według właściwości obiektu (wiek).

Kiedy warto używać sort()?

- ✓ Gdy chcesz **posortować tablicę** (liczby, stringi, obiekty).
- ✓ Gdy **potrzebujesz kontroli nad kolejnością** (rosnąco, malejąco, alfabetycznie).
- ✓ Gdy pracujesz z **danymi, które trzeba uporządkować** (np. lista użytkowników, produkty).

Kiedy NIE używać sort()?

- ✗ Jeśli chcesz zachować oryginalną tablicę – użyj **slice().sort()** zamiast.
- ✗ Jeśli chcesz tylko znaleźć największą wartość – użyj **Math.max(...tablica)**.
- ✗ Jeśli chcesz tylko znaleźć najmniejszą wartość – użyj **Math.min(...tablica)**.

Podsumowanie

- ♦ **Domyślnie sortuje jako stringi** – uważaj przy liczbach!
- ♦ **Zmienia oryginalną tablicę** – jeśli chcesz zachować starą, użyj slice().
- ♦ **Działa na liczbach, tekstach, obiektach** – jeśli podasz odpowiednią funkcję porównującą.

6. Odwracanie kolejności – reverse()

Metoda **reverse()** służy do **odwrócenia kolejności elementów w tablicy**.

Modyfikuje **oryginalną tablicę**, nie tworząc nowej.

Składnia

```
tablica.reverse();
```

- Nie przyjmuje żadnych argumentów.
- Zwraca odwróconą tablicę, ale też zmienia oryginalną tablicę.

Przykłady użycia reverse()

1. Odwrócenie kolejności elementów w tablicy

```
const liczby = [1, 2, 3, 4, 5];

liczby.reverse();

console.log(liczby); // [5, 4, 3, 2, 1]
```

Kolejność elementów została zmieniona w miejscu.

2. Odwrócenie kolejności znaków w stringu

Nie można użyć reverse() bezpośrednio na stringu, ale można połączyć go z split() i join().

```
const tekst = "JavaScript";

const odwroconyTekst = tekst.split("").reverse().join("");

console.log(odwroconyTekst); // "tpircSavaJ"
```

- split("") zamienia string na tablicę liter,
- reverse() odwraca tablicę,
- join("") łączy litery z powrotem w string.

3. Odwrócenie posortowanej tablicy (sortowanie malejące)

Jeśli chcesz posortować liczby **od największej do najmniejszej**, możesz użyć sort() + reverse().

```
const liczby = [3, 1, 4, 2, 5];

liczby.sort((a, b) => a - b).reverse();

console.log(liczby); // [5, 4, 3, 2, 1]
```

- sort((a, b) => a - b) najpierw sortuje rosnąco.
- reverse() odwraca kolejność, tworząc sortowanie malejące.

Kiedy warto używać reverse()?

- ✓ Gdy **chcesz odwrócić kolejność elementów** w tablicy.
- ✓ Gdy **potrzebujesz sortowania malejącego** (w połączeniu z sort()).
- ✓ Gdy **chcesz odwrócić tekst** (przy użyciu split().reverse().join()).

Kiedy NIE używać reverse()?

✗ Jeśli chcesz zachować oryginalną tablicę – użyj **slice().reverse()** zamiast.

```
const liczby = [1, 2, 3, 4, 5];

const odwrocone = [...liczby].reverse(); // lub liczby.slice().reverse()

console.log(odwrocone); // [5, 4, 3, 2, 1]
console.log(liczby); // [1, 2, 3, 4, 5] (oryginalna tablica niezmieniona)
```

Podsumowanie

- ♦ reverse() **zmienia oryginalną tablicę** – jeśli chcesz kopię, użyj slice().
- ♦ Używany do **odwracania kolejności elementów** w tablicy.
- ♦ Można go połączyć z sort() do **sortowania malejącego**.
- ♦ Może służyć do **odwracania stringów** (split().reverse().join()).